

GIAN LUCA POZZATO

INTELLIGENZA ARTIFICIALE E LABORATORIO

INTERPRETE PROLOG E RISOLUZIONE SLD

INTERPRETE PROLOG

- Analizziamo in dettaglio come avviene l'esecuzione dei programmi Prolog
- Esecuzione mediante backward chaining in profondità
- Si parte dal goal che si vuole derivare:
 - goal = congiunzione di formule atomiche $G_1, G_2, \dots, G_n$
 - Voglio dimostrare mediante risoluzione che il goal segue logicamente dal programma
- Una regola $A :- B_1, B_2, \dots, B_m$ è applicabile a G_i se:
 - le variabili vengono rinominate
 - A e G_i unificano

INTERPRETE PROLOG

FORMULAZIONE NON DETERMINISTICA

interprete (G)

if G vuoto

then successo

else sia $G = G_1, G_2, \dots, G_n$

 sia R l'insieme delle clausole applicabili a G_1

 if R vuoto

 then fallimento

 else scegliere una clausola $A :- B_1, B_2, \dots, B_m$ in R

 sia σ il MGU di G_1 e A

 interprete ($B_1\sigma, B_2\sigma, \dots, B_m\sigma, G_2\sigma, \dots, G_n\sigma$)

INTERPRETE PROLOG

- La computazione ha successo se esiste una computazione che termina con successo
- Non determinismo: non è specificata la regola scelta in R
- L'interprete Prolog si comporta in modo deterministico:
 - le clausole vengono considerate nell'ordine in cui sono scritte nel programma
 - viene fatto backtracking all'ultimo punto di scelta ogni volta che la computazione fallisce
- In caso di successo, l'interprete restituisce una sostituzione per le variabili che compaiono nel goal

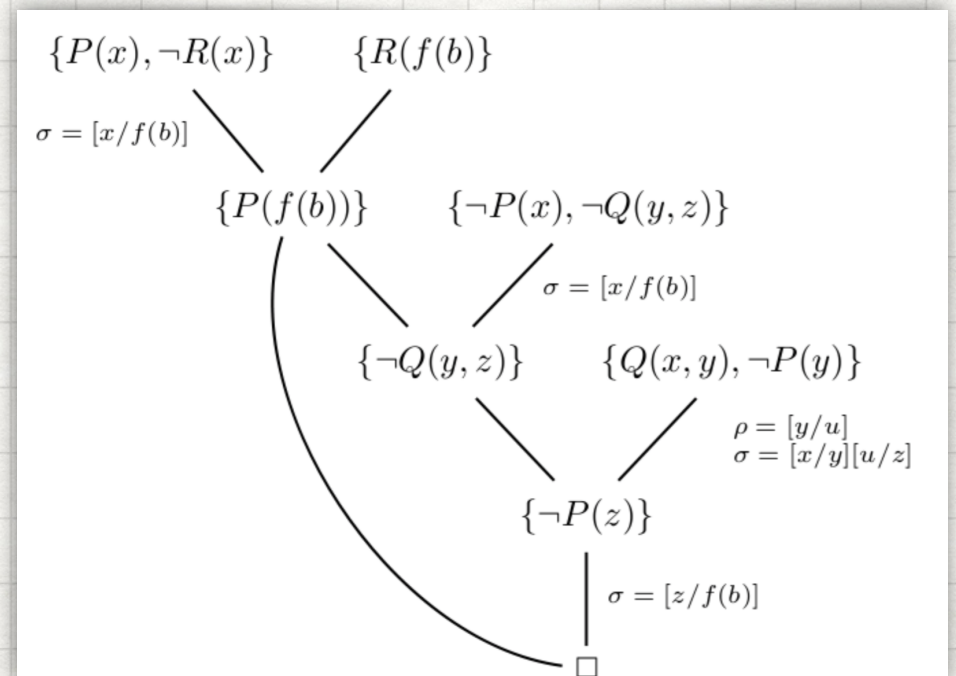
PERCHE' PARLIAMO
DI CONSEGUENZA
LOGICA DEL GOAL
DAL PROGRAMMA?

QUAL E' LA
RELAZIONE CON
LA LOGICA E LA
RISOLUZIONE?

```

1  /* imports */
2  :- use_module(library(lists)).
3  :- op(300,fy,!),
4  op(500,xfy,=>),
5  op(400,xfy,->),
6  op(400,xfy,^),
7  op(400,xfy,v).
8
9
10 prova(Formula):-prove([Formula],Tree),write(Tree),!.
11 prova(_):-write('Not Valid').
12
13
14 fillTheHole(NS,ListToFill,NewNS):-select(hole,NS,TempNS),!,
15                                append(TempNS,ListToFill,NewNS).
16 fillTheHole(NS,ListToFill,[[[Fml,NewGamma],AppliedConditionals]|TempNS]):-select([[Fml,Gamma],AppliedConditionals],NS,NewNS),
17                                fillTheHole(Gamma,ListToFill,NewGamma).
18
19 selectList([],NS,[hole|NS]).
20 selectList([Fml|Tail],NS,NewNS):-select(Fml,NS,TempNS),
21                                selectList(Tail,TempNS,NewNS).
22
23 deepSelect(List,NS,NewNS):-selectList(List,NS,NewNS).
24 deepSelect(List,NS,[[[Fml,NewGamma],AppliedConditionals]|NewNS]):-select([[Fml,Gamma],AppliedConditionals],NS,NewNS),
25                                deepSelect(List,Gamma,NewGamma).
26
27 memberList([],_).
28 memberList([Head|Tail],NS):-member(Head,NS),memberList(Tail,NS).
29 deepMember(List,NS):-memberList(List,NS).
30 deepMember(List,NS):-member([_,Gamma],_ ,NS),
31                                deepMember(List,Gamma).
32
33
34
35 prove(NS,tree(ax,NS,no,no,no)):-deepMember([P,!P],NS),!.
36 prove(NS,tree(agt,NS,no,no,no)):-deepMember([top],NS),!.
37 prove(NS,tree(axb,NS,no,no,no)):-deepMember([!bot],NS),!.
38 prove(NS,tree(neg,NS,SubTree,no,no)):-deepSelect([!A],NS,NewNS),!,fillTheHole(NewNS,[A],DefNS),prove(DefNS,SubTree),!.
39 prove(NS,tree(orp,NS,SubTree,no,no)):-deepSelect([A v B],NS,NewNS),!,fillTheHole(NewNS,[A,B],DefNS),prove(DefNS,SubTree),!.
40 prove(NS,tree(andn,NS,SubTree,no,no)):-deepSelect([!(A ^ B)],NS,NewNS),!,fillTheHole(NewNS,[A,!B],DefNS),prove(DefNS,SubTree),!.
41 prove(NS,tree(impn,NS,SubTree,no,no)):-deepSelect([A -> B],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,SubTree),!.
42 prove(NS,tree(condp,NS,SubTree,no,no)):-deepSelect([A => B],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,SubTree),!.
43 prove(NS,tree(orn,NS,Sub1,Sub2,no)):-deepSelect([!(A v B)],NS,NewNS),!,fillTheHole(NewNS,[!A,!B],DefNS),prove(DefNS,Sub1),!.
44 prove(NS,tree(impn,NS,Sub1,Sub2,no)):-deepSelect([A ^ B],NS,NewNS),!,fillTheHole(NewNS,[A,!B],DefNS),prove(DefNS,Sub1),!.
45 prove(NS,tree(impn,NS,Sub1,Sub2,no)):-deepSelect([!(A -> B)],NS,NewNS),!,fillTheHole(NewNS,[A,!B],DefNS),prove(DefNS,Sub1),!.
46 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
47 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
48 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
49 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
50 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
51 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
52 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
53 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
54 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
55 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[!A,B],DefNS),prove(DefNS,Sub1),!.
56 /* Auxiliary predicates for building the latex source file containing a derivation of a sequent */
57 getRuleName(tree(ax,...)):-write('AX'),!.

```



SEMANTICA E METODI DI PROVA

- Un brevissimo ripasso
 - logica classica
 - semantica vs metodi di prova
 - risoluzione
 - risoluzione SLD

LOGICA CLASSICA

- Conseguenza logica definita semanticamente
 - dato una teoria e una formula, diciamo che la formula segue dalla teoria se essa è vera in tutti i modelli della teoria
- Esempio:
 - Teoria K:
 - i gatti miagolano $\text{gatto} \rightarrow \text{miagola}$
 - i persiani sono gatti $\text{persiano} \rightarrow \text{gatto}$
 - Voglio dimostrare che "i persiani miagolano" è conseguenza logica della teoria K:

$$K \models \text{persiano} \rightarrow \text{miagola}$$

LOGICA CLASSICA

- Semantica: tavola di verità

[illegible]

LOGICA CLASSICA

- Semantica: tavola di verità

$gatto \rightarrow miagola$			$persiano \rightarrow gatto$				$persiano \rightarrow miagola$		
0	1	0	1	0	1	0	0	1	0
0	1	1	1	0	1	0	0	1	1
0	1	0	0	1	0	0	1	0	0
0	1	1	0	1	0	0	1	1	1
1	0	0	0	0	1	1	0	1	0
1	1	1	1	0	1	1	0	1	1
1	0	0	0	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1

$gatto \rightarrow miagola \wedge persiano \rightarrow gatto$

LOGICA CLASSICA

- Semantica: tavola di verità

<i>gatto</i> $\rightarrow$ <i>miagola</i>			<i>persiano</i> $\rightarrow$ <i>gatto</i>				<i>persiano</i> $\rightarrow$ <i>miagola</i>		
0	1	0	1	0	1	0	0	1	0
0	1	1	1	0	1	0	0	1	1
0	1	0	0	1	0	0	1	0	0
0	1	1	0	1	0	0	1	1	1
1	0	0	0	0	1	1	0	1	0
1	1	1	1	0	1	1	0	1	1
1	0	0	0	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1

$K \models \textit{persiano} \rightarrow \textit{miagola}$

In ogni modello di K la
formula *persiano* $\rightarrow$ *miagola*
è vera

LOGICA CLASSICA

- Piuttosto laborioso già con poche formule e teorie/basi di conoscenza piccole
- Metodo di prova:
 - procedura/algoritmo che calcola/dimostra se una formula è conseguenza logica della teoria
 - corretto: se l'algoritmo dimostra F da K, allora F è conseguenza logica di K
 - completo: se F è conseguenza logica di K, allora l'algoritmo dimostra F da K
- Esempio: dimostriamo con la risoluzione che

$$K \models \textit{persiano} \rightarrow \textit{miagola}$$

LOGICA CLASSICA

RISOLUZIONE

- Risoluzione:
 - si applica a formule in forma di clausole
 - clausola: disgiunzione di letterali
 - si basa su un'unica regola di inferenza:
 - date due clausole $C_1 = A_1 \vee \dots \vee A_n$ e $C_2 = B_1 \vee \dots \vee B_m$
 - se ci sono due letterali A_i e B_j tali che $A_i = \neg B_j$, allora posso derivare la clausola **risolvente** $A_1 \vee \dots \vee A_{i-1} \vee A_{i+1} \vee \dots \vee A_n \vee B_1 \vee \dots \vee B_{j-1} \vee B_{j+1} \vee \dots \vee B_m$
 - il risolvente è conseguenza logica di $C_1 \cup C_2$

LOGICA CLASSICA

RISOLUZIONE

- Data una teoria (=insieme di formule) K e una formula F , dimostro che F è conseguenza logica di K per refutazione:
 - dimostro che $K \cup \neg F$ è inconsistente
- Si parte dalle clausole $K \cup \neg F$, risolvendo ad ogni passo due clausole e aggiungendo il risolvente all'insieme di clausole
- Si conclude quando si ottiene la clausola vuota $\square$

LOGICA CLASSICA

K

$\neg \text{gatto} \vee \text{miagola}$

$\neg \text{persiano} \vee \text{gatto}$

$\neg(\text{persiano} \rightarrow \text{miagola})$

LOGICA CLASSICA

K

$\neg \text{gatto} \vee \text{miagola}$

$\neg \text{persiano} \vee \text{gatto}$

persiano

$\neg \text{miagola}$

$K \models \text{persiano} \rightarrow \text{miagola}$

Applico risoluzione a K + negazione della formula da provare e ottengo la clausola vuota

LOGICA CLASSICA

K

$\neg \text{gatto} \vee \text{miagola}$

$\neg \text{persiano} \vee \text{gatto}$

persiano

$\neg \text{miagola}$

$\neg \text{gatto}$



$K \models \text{persiano} \rightarrow \text{miagola}$

Applico risoluzione a K + negazione della formula da provare e ottengo la clausola vuota

LOGICA CLASSICA

K

$\neg \text{gatto} \vee \text{miagola}$

$\neg \text{persiano} \vee \text{gatto}$

persiano

$\neg \text{miagola}$

$\neg \text{gatto}$

gatto

$K \models \text{persiano} \rightarrow \text{miagola}$

Applico risoluzione a K + negazione della formula da provare e ottengo la clausola vuota

LOGICA CLASSICA

K

$\neg \text{gatto} \vee \text{miagola}$

$\neg \text{persiano} \vee \text{gatto}$

persiano

$\neg \text{miagola}$

$\neg \text{gatto}$

gatto



$K \models \text{persiano} \rightarrow \text{miagola}$

Applico risoluzione a K + negazione della formula da provare e ottengo la clausola vuota

LOGICA CLASSICA

RISOLUZIONE

- Risoluzione:
 - se le due clausole $C_1 = A_1 \vee \dots \vee A_n$ e $C_2 = B_1 \vee \dots \vee B_m$ contengono variabili, i due letterali A_i e B_j devono essere tali che si possa fare l'**unificazione** tra i due
 - unificazione: sostituzione σ di variabili con termini o uguaglianza di variabili affinché $A_i = \neg B_j$
 - clausola **risolvente** $[A_1 \vee \dots A_{i-1} \vee A_{i+1} \vee \dots \vee A_n \vee B_1 \vee \dots B_{j-1} \vee B_{j+1} \vee \dots \vee B_m] \sigma$
 - le sostituzioni di σ sono applicate a $A_1 \vee \dots A_{i-1} \vee A_{i+1} \vee \dots \vee A_n \vee B_1 \vee \dots B_{j-1} \vee B_{j+1} \vee \dots \vee B_m$

LOGICA CLASSICA

K

$\forall x(Chirurgo(x) \rightarrow Medico(x))$

$\forall x(Medico(x) \rightarrow Laureato(x))$

$Chirurgo(figlioDi(franco))$

LOGICA CLASSICA

K

$\neg \textit{Chirurgo}(x) \vee \textit{Medico}(x)$
 $\neg \textit{Medico}(x) \vee \textit{Laureato}(x)$
 $\textit{Chirurgo}(\textit{figlioDi}(\textit{franco}))$

K $\models \textit{Laureato}(\textit{figlioDi}(\textit{franco}))$

LOGICA CLASSICA

$$\neg \textit{Medico}(x) \vee \textit{Laureato}(x)$$

$$\neg \textit{Laureato}(\textit{figlioDi}(\textit{franco}))$$

$$\textit{Chirurgo}(\textit{figlioDi}(\textit{franco}))$$

$$\neg \textit{Chirurgo}(x) \vee \textit{Medico}(x)$$

LOGICA CLASSICA

$$\neg \textit{Medico}(x) \vee \textit{Laureato}(x)$$

$$\neg \textit{Laureato}(\textit{figlioDi}(\textit{franco}))$$

$x / \textit{figlioDi}(\textit{franco})$

$$\neg \textit{Medico}(\textit{figlioDi}(\textit{franco}))$$

$$\textit{Chirurgo}(\textit{figlioDi}(\textit{franco}))$$

$$\neg \textit{Chirurgo}(x) \vee \textit{Medico}(x)$$

LOGICA CLASSICA

$$\neg \textit{Medico}(x) \vee \textit{Laureato}(x)$$

$$\neg \textit{Laureato}(\textit{figlioDi}(\textit{franco}))$$

$x / \textit{figlioDi}(\textit{franco})$

$$\neg \textit{Medico}(\textit{figlioDi}(\textit{franco}))$$

$$\textit{Chirurgo}(\textit{figlioDi}(\textit{franco}))$$

$x / \textit{figlioDi}(\textit{franco})$

$$\neg \textit{Chirurgo}(x) \vee \textit{Medico}(x)$$

$$\textit{Medico}(\textit{figlioDi}(\textit{franco}))$$

LOGICA CLASSICA

$\neg \text{Medico}(x) \vee \text{Laureato}(x)$

$\neg \text{Laureato}(\text{figlioDi}(\text{franco}))$

$x / \text{figlioDi}(\text{franco})$

$\neg \text{Medico}(\text{figlioDi}(\text{franco}))$

$\text{Chirurgo}(\text{figlioDi}(\text{franco}))$

$x / \text{figlioDi}(\text{franco})$

$\neg \text{Chirurgo}(x) \vee \text{Medico}(x)$

$\text{Medico}(\text{figlioDi}(\text{franco}))$



LOGICA CLASSICA

UNIFICAZIONE DI DUE TERMINI

	costante c_2	variabile x_2	composto s_2
costante c_1	unificano se $c_1 = c_2$	unificano con x_2/c_1	non unificano
variabile x_1	unificano con x_1/c_2	unificano con x_1/x_2	unificano con x_1/s_2
composto s_1	non unificano	unificano con x_2/s_1	unificano se il funtore in s_1 e s_2 è lo stesso e gli argomenti unificano

- NOTA: per ragioni d'efficienza, Prolog non fa occur check, ossia una variabile X unifica con $f(X)$
 - strategia non corretta

LOGICA CLASSICA

RISOLUZIONE SLD

- Per arrivare ad un linguaggio di programmazione (Prolog) vogliamo una strategia efficiente
- Risoluzione SLD: **L**inear resolution with **S**election function for **D**efinite clauses
 - K con clausole DEFINITE
 - clausole di Horn = al più un letterale non negato
 - strategia linear input: ad ogni passo di risoluzione, una **variante** di una clausola è sempre scelta nella K di partenza (programma) mentre l'altra è sempre il risolvente del passo precedente (goal, la negazione di F al primo passo)
 - variante: clausola con variabili rinominate
- Strategia NON completa in generale, completa per clausole di Horn

LOGICA CLASSICA

RISOLUZIONE SLD

- Derivazione SLD per un goal G_0 da un insieme di clausole K è
 - una sequenza di clausole goal $G_0, G_1, \dots, G_n$
 - una sequenza di varianti di clausole di K $C_1, C_2, \dots, C_n$
 - una sequenza di MGU $\sigma_1, \sigma_2, \dots, \sigma_n$, tali che G_{i+1} è derivato da G_i e da C_{i+1} attraverso la sostituzione σ_{i+1} ,
- Tre possibili tipi di derivazioni:
 - successo se $G_n = \square$
 - fallimento finito, se non è possibile derivare da G_n alcun risolvibile e $G_n \neq \square$
 - fallimento infinito, se è sempre possibile derivare nuovi risolventi

LOGICA CLASSICA

RISOLUZIONE SLD

- Due forme di non determinismo:
 - regola di calcolo per selezionare ad ogni passo l'atomo B_i del goal da unificare con una clausola
 - scelta di quale clausola utilizzare ad ogni passo di risoluzione
- Regola di calcolo: funzione che ha come dominio l'insieme dei goal e per ogni goal seleziona un suo atomo
 - non influenza correttezza e completezza del metodo di prova

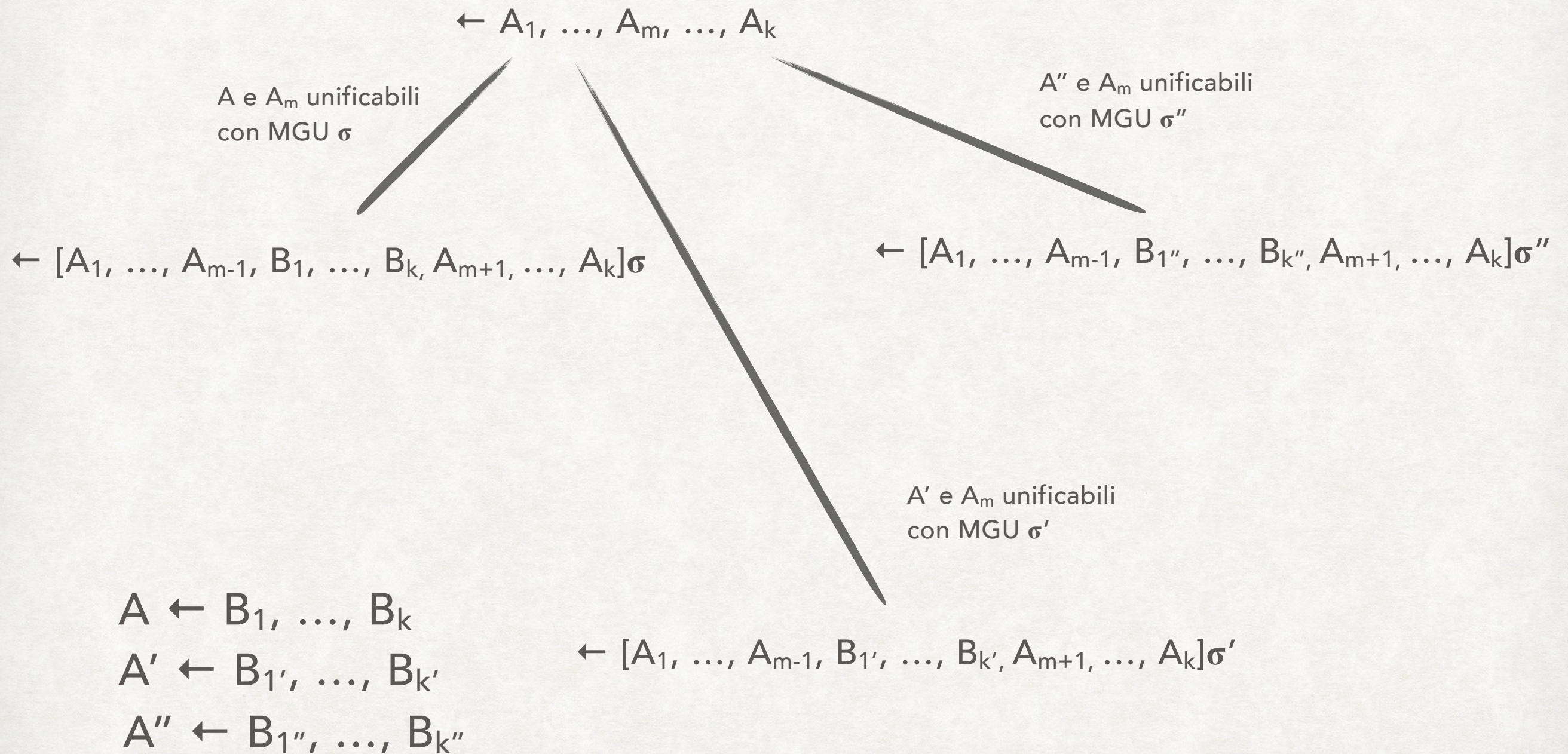
LOGICA CLASSICA

RISOLUZIONE SLD

- Data una regola di calcolo, è possibile rappresentare tutte le derivazioni con un albero SLD:
 - nodo: goal
 - radice: goal iniziale G_0
 - ogni nodo $\leftarrow A_1, \dots, A_m, \dots, A_k$, dove A_m è l'atomo selezionato dalla regola di calcolo, ha un figlio per ogni clausola $A \leftarrow B_1, \dots, B_k$ tale che A e A_m sono unificabili con MGU σ . Il nodo figlio è etichettato con il goal $\leftarrow [A_1, \dots, A_{m-1}, B_1, \dots, B_k, A_{m+1}, \dots, A_k]\sigma$. Il ramo dal padre al figlio è etichettato con σ e con la clausola selezionata

LOGICA CLASSICA

RISOLUZIONE SLD



RISOLUZIONE SLD

ESEMPIO

$C_1 \quad \forall x (PosaFacile(x) \wedge Economico(x) \rightarrow Acquisto(x))$

$C_2 \quad \forall x (InOfferta(x) \rightarrow Acquisto(x))$

$C_3 \quad \forall x (InMagazzino(x) \rightarrow Disponibile(x))$

$C_4 \quad InOfferta(mosaico)$

$C_5 \quad Disponibile(laminato)$

$C_6 \quad PosaFacile(laminato)$

$C_7 \quad Economico(laminato)$

GOAL $\quad Disponibile(x), Acquisto(x)$

RISOLUZIONE SLD

ESEMPIO

$C_1 \quad \neg PosaFacile(x) \vee \neg Economico(x) \vee Acquistato(x)$

$C_2 \quad \neg InOfferta(x) \vee Acquistato(x)$

$C_3 \quad \neg InMagazzino(x) \vee Disponibile(x)$

$C_4 \quad InOfferta(mosaico)$

$C_5 \quad Disponibile(laminato)$

$C_6 \quad PosaFacile(laminato)$

$C_7 \quad Economico(laminato)$

GOAL $\neg Disponibile(x) \vee \neg Acquistato(x)$

RISOLUZIONE SLD

ALBERO SLD CON REGOLA LEFTMOST

$\neg \textit{Disponibile}(x) \vee \neg \textit{Acquisto}(x)$

C_3

C_5

$\sigma = \{x/\textit{laminato}\}$

$\neg \textit{InMagazzino}(x) \vee \neg \textit{Acquisto}(x)$

Fallimento

$\neg \textit{Acquisto}(\textit{laminato})$

$C_1 \quad \sigma = \{x_1/\textit{laminato}\}$

$C_2 \quad \sigma = \{x_1/\textit{laminato}\}$

$\neg \textit{PosaFacile}(\textit{laminato}) \vee \neg \textit{Economico}(\textit{laminato})$

C_6

$\neg \textit{Economico}(\textit{laminato})$

C_7



$\neg \textit{InOfferta}(\textit{laminato})$

Fallimento

RISOLUZIONE SLD

ESEMPIO

$C_1 \quad \neg PosaFacile(x) \vee \neg Economico(x) \vee Acquisto(x)$

$C_2 \quad \neg InOfferta(x) \vee Acquisto(x)$

$C_3 \quad \neg InMagazzino(x) \vee Disponibile(x)$

$C_4 \quad InOfferta(mosaico)$

$C_5 \quad Disponibile(laminato)$

$C_6 \quad PosaFacile(laminato)$

$C_7 \quad Economico(laminato)$

GOAL $\neg Disponibile(x) \vee \neg Acquisto(x)$

RISOLUZIONE SLD

ALBERO SLD CON REGOLA RIGHTMOST

$\neg \text{Disponibile}(x) \vee \neg \text{Acquisto}(x)$

C₁

C₂

$\neg \text{Disponibile}(x) \vee \neg \text{InOfferta}(x)$

$\sigma = \{x/\text{mosaico}\}$

C₄

$\neg \text{Disponibile}(x) \vee \neg \text{PosaFacile}(x) \vee \neg \text{Economico}(x)$

C₇ $\sigma = \{x/\text{laminato}\}$

$\neg \text{Disponibile}(\text{laminato}) \vee \neg \text{PosaFacile}(\text{laminato})$

C₆

$\neg \text{Disponibile}(\text{laminato})$

C₅



C₃ $\sigma = \{x_1/\text{laminato}\}$

$\neg \text{InMagazzino}(\text{laminato})$

Fallimento

$\neg \text{Disponibile}(\text{mosaico})$

C₃ $\sigma = \{x_1/\text{mosaico}\}$

$\neg \text{InMagazzino}(\text{mosaico})$

Fallimento

RISOLUZIONE SLD E PROLOG

- Una computazione in Prolog corrisponde a una dimostrazione mediante risoluzione SLD
- Scelte per rendere la strategia deterministica:
 - regola di computazione: **leftmost**
 - viene sempre scelto il sottogoal più a sinistra
 - clausole considerate nell'**ordine in cui sono scritte nel programma**
 - strategia di ricerca: **in profondità con backtracking**
 - NON COMPLETA: se una computazione che porta al successo si trova a destra di un ramo infinito, l'interprete non lo trova perché entra, senza mai uscirne, nel ramo infinito

RISOLUZIONE SLD

PROLOG: LEFTMOST + DEPTH FIRST + BACKTRACKING

$\neg \text{Disponibile}(x) \vee \neg \text{Acquisto}(x)$

1

C_3

C_5

$\sigma = \{x/\text{laminato}\}$

$\neg \text{InMagazzino}(x) \vee \neg \text{Acquisto}(x)$

backtracking

$C_1 \quad \sigma = \{x_1/\text{laminato}\}$

3

$\neg \text{PosaFacile}(\text{laminato}) \vee \neg \text{Economico}(\text{laminato})$

C_6

4

$\neg \text{Economico}(\text{laminato})$

C_7



successo

$\neg \text{Acquisto}(\text{laminato})$

$C_2 \quad \sigma = \{x_1/\text{laminato}\}$

$\neg \text{InOfferta}(\text{laminato})$

Fallimento

IN PROLOG

ESEMPIO

C₁ acquisto(X) :- posaFacile(X), economico(X).

C₂ acquisto(X) :- inOfferta(X).

C₃ disponibile(X) :- inMagazzino(X).

C₄ inOfferta(mosaico).

C₅ disponibile(laminato).

C₆ posaFacile(laminato).

C₇ economico(laminato).

GOAL disponibile(X), acquisto(X).

RISOLUZIONE SLD

PROLOG: LEFTMOST + DEPTH FIRST + BACKTRACKING

